

StatGuard Agent: An auditable statistical analysis framework pairing LLM orchestration with a deterministic, cross-validated statistics engine

Cheng Peng¹

¹ Independent Researcher

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) ↗
- [Repository](#) ↗
- [Archive](#) ↗

Editor: [Open Journals](#) ↗

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

StatGuard Agent is a Python framework for conducting statistical analysis from natural-language requests while guaranteeing that every reported statistic is deterministic, reproducible, and independently verified. It is built on a strict separation of responsibilities: a large language model (LLM) acts only as an *orchestrator* that interprets the request and selects which analysis to run, while all statistical computation is performed by a fixed library of plugins whose numerical output is cross-validated against `scipy` and `statsmodels` (Seabold & Perktold, 2010; Virtanen et al., 2020). The LLM never computes a statistic and cannot alter a numerical result.

This design targets a specific failure mode of using LLMs as data analysts: the same request, on the same data, can produce different statistical methods and different conclusions on different runs. By confining the LLM to routing and delegating all computation to deterministic, pre-validated methods, StatGuard Agent produces analyses that are identical across runs and auditable down to the individual reported number.

Statement of need

LLMs are increasingly used to perform data analysis by generating and executing code. Their statistical output, however, is stochastic and difficult to reproduce: recent empirical work shows that the same prompt and data can yield materially different analytical pathways and conclusions across independent runs (Cui & Alexander, 2026), and that reproducibility of LLM-generated data-science workflows is generally low (Zeng et al., 2025). For exploratory work this variability may be acceptable; for research, clinical, or regulatory analysis — settings where a reported p-value or effect size must be defensible and reproducible — it is not.

A parallel line of work argues that LLM-driven scientific workflows must be constrained for reproducibility and auditability, and proposes execution-level frameworks that enforce deterministic, traceable execution of agent actions (Sureshkumar, 2026). These frameworks establish that an analysis can be *replayed*; they do not, however, verify that the statistics produced are *correct* — that the appropriate test was chosen, that its assumptions were checked, and that the resulting numbers match an independent reference implementation.

StatGuard Agent addresses this gap for the specific domain of statistical inference. Its contribution is not the general idea of constraining an LLM, but the combination of (1) deterministic, assumption-aware statistical methods that encode the kind of decisions a statistician makes, and (2) a validation regime that checks those methods' numerical output, case by case, against reference implementations. The framework is intended for researchers, graduate students, and analysts who need LLM-level convenience in expressing an analysis but require statistical results that are reproducible and independently verifiable — a combination

not offered by general-purpose LLM tools (which compute statistics stochastically) nor by traditional statistical software such as SPSS or base `scipy/statsmodels` (which are reproducible but require the user to specify the method and check assumptions themselves).

Design and key features

LLM orchestration, deterministic computation. A supervisor LLM selects one analysis action at a time from a registry of 25 tool plugins. Each plugin implements a hardcoded statistical method; the same data and arguments always produce identical output. The LLM's role is limited to choosing which plugin to invoke.

Assumption-aware routing. Method selection is governed by explicit, inspectable rules rather than left to the LLM. For example, the group-comparison plugin checks variance homogeneity (Levene's test) to route between Welch's t-test and classical procedures, and switches from a parametric test to a rank-based non-parametric alternative when normality is violated together with a small sample (group $n < 30$) or strong skew ($|\text{skewness}| \geq 1.5$).

Independent cross-validation. A 246-case benchmark (`benchmark/carpet/`) generates statistical scenarios with fixed random seeds and compares each plugin's output against an independent `scipy/statsmodels` computation of the same quantity. All 246 cases currently pass, providing case-by-case evidence that the framework's reported statistics are numerically correct.

Anti-fabrication claims ledger. Plugins emit structured, verified claims; the LLM may only reference these claims by identifier, and a render layer substitutes the verified wording into the final report. This prevents the LLM from reporting a statistic that was not actually computed.

Reproducibility and provenance. Analyses are tied to an explicit data version, and reports record how the analysis-ready dataset was constructed, including SQL provenance where applicable. The framework also exports a reproducibility manifest and APA-style methods text.

Testing. The framework includes 615 deterministic unit and integration tests that run without an API key or network access, in addition to the 246-case numerical validation benchmark.

Acknowledgements

The author thanks the maintainers of `scipy`, `statsmodels`, and `LangGraph`, on which this framework builds.

References

- Cui, J., & Alexander, R. (2026). *Same prompt, different outcomes: Evaluating the reproducibility of data analysis by LLMs*. <https://arxiv.org/abs/2602.14349>
- Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and statistical modeling with Python. *Proceedings of the 9th Python in Science Conference*, 92–96. <https://doi.org/10.25080/Majora-92bf1922-011>
- Sureshkumar, S. (2026). *R-LAM: Reproducibility-constrained large action models for scientific workflow automation*. <https://arxiv.org/abs/2601.09749>
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., & others. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3), 261–272. <https://doi.org/10.1038/s41592-019-0686-2>
- Zeng, Q., Jin, C., Wang, X., Zheng, Y., & Li, Q. (2025). *AIRepr: An analyst-inspector framework for evaluating reproducibility of LLMs in data science*. <https://arxiv.org/abs/>

DRAFT